AICTE VERIFIED STUDENT REGISTER

Authentication and Authorization Specification

User accounts, sessions, permissions, API boundaries and security tests for all eight roles.

Document: 03-AUTH Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Terms

Term	Meaning	Example
Authentication	Proves the user controls a valid account.	An operator logs in with email and password.
Authorization	Decides which actions and data that user may access.	The operator can upload but cannot certify.
Role-based access control (RBAC)	Permissions grouped by job role.	Only COLLEGE_HEAD may certify a batch.
Data scope	Limits which institution, region, student or case is visible.	A regional officer sees only the assigned region.
Session	Server-tracked login state presented by a browser cookie.	Logout invalidates the session server-side.

2. Proposed authentication approach

Use server-side sessions for the initial same-site React/Express prototype. Recommended packages: express-session with a MongoDB session store such as connect-mongo, plus bcrypt for password hashing. These are proposed engineering choices for the MERN version; they are not named in the source report.

1. User submits email and password over HTTPS in a production environment (localhost is allowed for development).

2. Server normalizes the account identifier, retrieves the active user, and compares the supplied password with the stored hash.

3. If valid, regenerate the session ID to reduce session-fixation risk and store only the minimum user reference/session metadata.

4. Set an HttpOnly cookie with an appropriate SameSite policy; use Secure in HTTPS deployments. Keep the actual session data server-side.

5. For each protected request, middleware loads the authenticated user and checks active status, role and data scope.

6. Logout invalidates the server-side session and clears the browser cookie.

7. Session expiration, account disablement and password reset must invalidate relevant sessions according to documented policy.

Do not mix authentication with frontend state
A hidden button or a React route guard is not a security boundary. Every protected Express route and sensitive service operation must verify identity, role, scope and business preconditions.

3. Required roles

Role enum	Account scope	Core authority
COLLEGE_OPERATOR	One assigned institution	Create/upload batches; correct staging rows; view own institution seat ledger and conflicts.
COLLEGE_HEAD	One assigned institution	Review/return/certify its institution batches; handle institution-side conflict actions.
AICTE_NATIONAL_OFFICER	National scope	View national dashboards; review/assign/escalate risk and conflict cases; view permitted audit/report data.
AICTE_REGIONAL_OFFICER	One or more assigned regions	View regional dashboards and handle cases only within assigned region.
AICTE_SYSTEM_ADMIN	System administration scope	Manage accounts/role assignments and draft/publish policy rules through maker-checker controls.
DEGREE_AWARDING_BODY	Assigned issuer organisation	View eligible records and issue/status credentials within issuer permissions.
VERIFIER	Registered verifier organisation	Create verification requests and view only results authorised for that request.
STUDENT	Own student reference only	View own record; approve/deny/revoke consent; view access history and own credential status.

4. Permission matrix

Capability	Operator	College Head	Regional	National	System Admin	Awarding Body	Verifier	Student
Upload/edit staging	Yes	Review only	No	No	No	No	No	No
Certify batch	No	Own college	No	No	No	No	No	No
View seat ledger	Own college	Own college	Assigned region summaries	National scope as permitted	Config/admin views	No by default	No	No
Resolve institution conflict	Own-side details / submit facts	Own college actions	Region case review	National escalation/review	No direct resolution by default	No	No	No
Manage users/roles	No	No	No	No	Yes	No	No	No
Change/publish seat policy	No	No	No	Review/oversight if granted	Draft and publish with separation of duties	No	No	No
Review risk case	No	No	Assigned region	National scope	No direct adjudication by default	No	No	No
Request verification	No	No	No	Legal mandate only if configured	No	No	Yes, own requests	No
Approve own consent	No	No	No	No	No	No	No	Yes
View own access history	No	No	No	No	No	No	No	Yes
Issue credential	No	No	No	Admin oversight only if approved	Policy/config only	Assigned issuer scope	No	Request/download if eligible
Public QR status	Public minimal status endpoint	Same	Same	Same	Same	Same	Same	Same

This permission matrix is the proposed implementation contract. If a source requirement does not establish a permission, the safer default is deny until the owner approves it.

5. Minimum auth endpoints

Method / path	Purpose	Access / expected outcome
POST /api/auth/login	Authenticate credentials and establish session.	Public with login rate limit; generic error on invalid credentials.
POST /api/auth/logout	Invalidate current session.	Authenticated; idempotent logout response.
GET /api/auth/me	Return current user reference, role and scope.	Authenticated; do not return password hash or secrets.
POST /api/auth/change-password	Change own password.	Authenticated; verify current password and rotate session as appropriate.
POST /api/admin/users	Create a managed user.	System admin only; allow-list role values; audit actor and changes.
PATCH /api/admin/users/:id	Change permitted user status/scope.	System admin only; prevent unsafe self-escalation and audit changes.
POST /api/admin/users/:id/password-reset	Initiate controlled reset.	System admin; never reveal an existing password or set a shared permanent password.

6. Login UI requirements

Control	Expected behaviour	Failure behaviour
Email/username field	Required, trim and normalize consistently.	Show a field-level message; never echo sensitive auth details.
Password field	Required, masked by default.	Do not log the password.
Show password toggle	Temporarily switch input visibility.	Must not send data to server until form submission.
Sign in	Submit once; show loading state.	Prevent duplicate submissions; show generic invalid-credentials message.
Forgot password	Only display if recovery is actually implemented.	Do not show a dead button. Demo may direct user to the documented admin reset process.
Sign out	Invalidate server session and return to login.	If network fails, show honest status; do not claim logout succeeded without server confirmation.

7. Security requirements

- Hash passwords with a suitable password-hashing library; never encrypt a password for later recovery and never store plaintext.
- Use a server-side session store, rotate session ID on login, set secure cookie options, configure expiration and invalidate sessions on logout.
- Use CSRF protection for cookie-authenticated state-changing requests. The coding agent must select and document a maintained implementation compatible with the app.
- Use strict CORS only if frontend/backend origins differ; do not use wildcard origins with credentialed requests.
- Rate-limit login and public verification endpoints; implement safe error messages and audit security-relevant actions.
- Do not permit self-service role selection or privileged public registration.
- Create demo users with synthetic accounts and hashed password fixtures; never commit production credentials.
- Disable account access when a user is inactive or disabled, even if a session cookie still exists.

8. Mandatory auth tests

- Valid login succeeds; wrong password and unknown user fail with no account enumeration.
- Unauthenticated access to protected endpoint returns 401.
- Authenticated user without permission receives 403.
- College A cannot read or mutate College B records by changing IDs in URL/body.
- Regional officer cannot access outside assigned region.
- Operator cannot call certification endpoint directly.
- Student cannot approve consent for another student.
- Disabled user and expired session are rejected.
- Logout invalidates the session.
- Login throttling and CSRF protection have explicit tests.